

QuickNotes

A Serverless Cloud Note Management Platform on AWS

Course	Cloud Computing — Final Project Report
Submitted By	Rayaan Hassan — 2023598
Team Member	Murtaza Ali Khan — 2023472
Submission Date	11 May 2026
Submission	Individual — One Submission Per Group

Cloud Computing Course — Final Project Report

1. Executive Summary

QuickNotes is a fully serverless, cloud-native note management platform built on six integrated Amazon Web Services. The platform enables users to create, retrieve, update, delete, and search personal notes with optional file attachments through a live web application. There is no traditional server — all compute, storage, and networking is managed by AWS.

AWS Services	Lambda Functions	REST Endpoints	Monthly Cost	Architecture	Servers Managed
6	8	9	~\$0	Serverless	Zero

Table 1: Project Key Performance Indicators

2. Project Overview & Real-World Relevance

The core problem QuickNotes addresses is genuine: professionals and students need a lightweight, cloud-hosted repository for notes and file attachments accessible from any browser. QuickNotes demonstrates the architectural foundation of exactly such a system — how a secured, event-driven cloud backend is designed and deployed using AWS managed services.

In real-world terms, QuickNotes implements patterns used directly in production cloud applications:

- Remote accessibility — all data resides on AWS, accessible from any device with no local storage dependency.
- Secure file association — attaching documents to records via pre-signed URLs mirrors enterprise systems such as SharePoint Online and Dropbox Business.
- Event-driven notifications — the SNS integration pattern is used by AWS itself and companies such as Uber and Airbnb for real-time operational alerts.
- Operational monitoring — automated CloudWatch alarms mirror how production engineering teams detect and respond to service degradation without manual intervention.

QuickNotes differs from standard course assignments in three measurable ways. First, it integrates six AWS services into one coherent request flow rather than demonstrating one service in isolation. Second, file handling follows the production-grade pre-signed URL pattern — Lambda never touches file bytes — which is the approach used by Dropbox, Google Drive, and S3-native applications. Third, the notification layer is decoupled through SNS, meaning Lambda publishes events that any number of downstream consumers can receive without any code change.

3. Technical Architecture

3.1 System Architecture Diagram

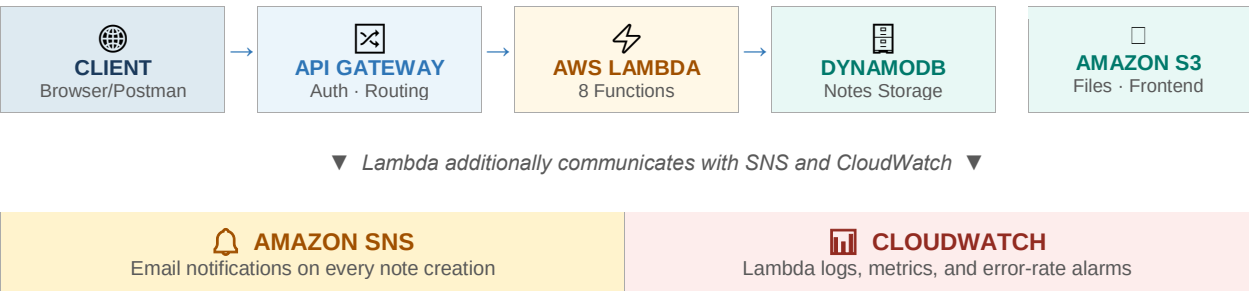


Figure 1: QuickNotes System Architecture

3.2 Request Lifecycle

Every request follows a deterministic, stateless path. A browser sends an HTTPS request to API Gateway with an API key header. API Gateway validates the key — rejecting invalid requests with 403 before any Lambda invocation. It then forwards the request as a Lambda proxy event to the correct function. Lambda executes the operation, reads or writes DynamoDB, and optionally invokes S3 or SNS. It returns a JSON response that API Gateway forwards to the client with CORS headers.

File operations differ: Lambda generates a short-lived pre-signed URL and returns it to the client. The client then communicates directly with S3 using that URL — Lambda is not involved in the file transfer. This eliminates Lambda memory constraints on file size, reduces execution time, and lowers cost. Pre-signed URLs expire after 300 seconds ensuring files cannot be accessed indefinitely.

3.3 Security Design

All Lambda functions share a single IAM execution role, `LambdaNotesRole`, with least-privilege policies: DynamoDB read/write, S3 pre-signed URL generation, SNS publish rights, and CloudWatch log writing. API key authentication is enforced at the API Gateway layer on every method. The S3 bucket blocks all public access — objects are accessible only through time-limited pre-signed URLs generated by Lambda.

4. AWS Services in Detail

Service	Role in QuickNotes	Key Pattern
API Gateway	Exposes all nine REST endpoints over HTTPS. Validates the API key on every request and routes to the correct Lambda function via proxy integration.	API Key · CORS · Routing
AWS Lambda	Executes all business logic as eight independent stateless Python 3.12 functions — one per operation. No server to manage or scale.	Event-driven · Python 3.12
Amazon DynamoDB	Serverless NoSQL database. Stores all note records by UUID partition key including title, content, timestamps, and S3 file reference.	NoSQL · Auto-scaling
Amazon S3	Stores all file attachments privately. Lambda generates a pre-signed URL; the client uploads or downloads directly to S3 — Lambda never handles file bytes.	Pre-signed URLs · Private
Amazon SNS	Decoupled pub/sub notification layer. Lambda publishes one event to the topic on note creation; SNS fans it to all confirmed subscribers via email.	Pub/Sub · Decoupled
CloudWatch	Full observability. Lambda invocations are logged automatically. A custom alarm monitors error rates and triggers SNS alert if errors exceed 5 per minute.	Alarms · Log Groups

Table 2: AWS Services — roles and key patterns

Amazon SNS warrants specific justification. Rather than calling an email service directly from Lambda, the system publishes to an SNS topic. This decoupling means Lambda always succeeds regardless of notification delivery, additional subscribers such as SMS or mobile push can be added without modifying application code, and the pattern is fully consistent with AWS-recommended event-driven architecture used in commercial systems managing millions of daily notifications.

5. API Endpoints

Method	Endpoint	Function	Description
POST	/notes	<code>createNote</code>	Creates a note. Triggers SNS email notification.
GET	/notes	<code>listNotes</code>	Returns all notes sorted newest first.

GET	/notes/{id}	getNote	Retrieves a single note by UUID.
PUT	/notes/{id}	updateNote	Updates title and/or content of a note.
DELETE	/notes/{id}	deleteNote	Permanently deletes a note from DynamoDB.
GET	/notes/search?search=kw	searchNotes	Case-insensitive keyword search across all notes.
GET	/notes/{id}/upload-url	getUploadUrl	Returns a pre-signed S3 PUT URL for file upload.
PUT	S3 Pre-signed URL	Direct to S3	Client uploads file directly to S3.
GET	/notes/{id}/download-url	getDownloadUrl	Returns a pre-signed S3 GET URL to retrieve file.

Table 3: Complete API endpoint reference

The search endpoint uses Python-level filtering rather than DynamoDB's native contains operator, which is case-sensitive. Both stored values and the search term are converted to lowercase before comparison, ensuring searches for "meeting" correctly match notes titled "Meeting Agenda" — consistent, user-friendly behaviour that DynamoDB alone cannot provide.

6. Testing & Validation

6.1 Unit & Integration Testing

Each Lambda function was tested in isolation in the AWS Lambda console before API Gateway connection. Test events simulated pathParameters, body, and queryStringParameters as injected by proxy integration. A Postman collection then tested all nine endpoints sequentially against the live deployment: create note and verify 201 with notelId; list notes and confirm the new note appears; retrieve by ID; update and verify the updatedAt timestamp changes; search by keyword and confirm only matching notes return; generate an upload URL, upload a file directly to S3, and confirm the file lands in the bucket; generate a download URL and confirm retrieval; delete and confirm absence.

6.2 Error Case Testing

The following error conditions were verified: missing title or content on POST returns 400; invalid API key returns 403 at API Gateway before any Lambda invocation; non-existent notelId returns 404; expired pre-signed URL returns 403 from S3; empty search query returns 400 with usage guidance.

6.3 CloudWatch Alarm Validation

Six consecutive Lambda errors were triggered deliberately using empty test events. The CloudWatch alarm transitioned from OK to In Alarm within 90 seconds and an SNS notification was delivered to the subscribed email, confirming the complete monitoring and alerting pipeline was operational.

7. Challenges & Solutions

Challenge	Solution
Lambda proxy integration not enabled on API Gateway	Identified via CloudWatch logs. Enabled proxy integration on each method and redeployed prod stage.
DynamoDB partition key case mismatch (noteId vs notelId)	Deleted and recreated the table with the correct camelCase key matching Lambda code.
CORS blocking requests from S3-hosted frontend	Re-enabled CORS on all API Gateway resources, confirmed OPTIONS methods, and redeployed.
DynamoDB contains operator is case-sensitive	Replaced FilterExpression with Python-level filtering converting all text to lowercase before comparison.

Pre-signed URL expiry during upload testing

Always generate a fresh URL immediately before use. URLs expire after 300 seconds by design.

Table 4: Technical challenges encountered and solutions implemented

8. Conclusion

QuickNotes delivered a fully operational, publicly accessible cloud application built on six AWS services with nine tested API endpoints, a live web frontend, real-time email notifications, and automated error monitoring. The system went beyond the original proposal scope by adding file download, case-insensitive search, a CloudWatch alarm, and a deployed frontend with an architecture dashboard.

The architectural decisions made throughout — pre-signed URLs for file handling, decoupled SNS notifications, least-privilege IAM roles, API key authentication, and automated CloudWatch monitoring — are genuine production patterns used in commercial AWS deployments. QuickNotes represents a faithful, working implementation of how modern serverless cloud applications are designed, secured, and operated, demonstrating mastery of multi-service cloud integration, REST API design, cloud security, and operational observability applied together in a single cohesive system.
